

Assignment-01

1- squash

(use commit , but meld into pervious commit)

بمعنى ان ادمج اكثر من commit مع بعض داخل commit واحدة فقط
مثال :

A→B→C→D→E↖Head

لو افترضنا ان الخمسة commit مرتبطين بنفس ال feature , ممكن ادمج اخر 4 commit مع بعض بحيث :

A→X

نقدر نعمل كده من خلال استخدام command التالى :

git rebase -i HEAD~n (n هي عدد ال :)

هيفتح شاشة فيها 4commit :

Pick B

Pick C

Pick D

Pick E

هنعدل اخر 3pick to squash

Pick B

Squash C

Squash D

Squash E

هيدمج C,D,E بداخل B فى النهاية :

A→B (ONLY 2 COMMIT)

**ال squash بيغير ال history وكم ان بيغير ال ID الخاص بال commit , بيكرت ID جديد خاص بال commit الجديدة

** النقطة دى ممكن تسبب مشاكل لو commit مرفوعة remote

** الافضل استخدام امر ال squash قبل ال pull request

2/ merge vs rebase

****Merge**

(main) A→B→C

↓D→E (feature)

هنا عايزة اضيف ال commit الخاصة بال feature branch على ال main branch هنعمل كده بطريقتين :
اول طريقة وهى ال mrege :

git switch main
git merge feature

(main) $A \rightarrow B \rightarrow C \rightarrow M$
 $\downarrow D \rightarrow E \nearrow$

عملنا commit جديدة والميرج تم بطريقة 3way merge
**الميزة هنا ان history التفرع متغيرش على ال workflow فقط تم اضافة نقطة commit جديدة

****rebase :**

تانى طريقة هي استخدام :

git switch feature
git rebase main

$A \rightarrow B \rightarrow C \rightarrow D' \rightarrow E'$

فى الحالة دى git بياخد ال commits الخاصة بال feature branch ويضعها دايركت على ال main branch
** لكن هنا يتم تغير ال IDs الخاصة بال commit وكذلك تغيير ال history
**ال rebase مفيدة فى البروجكت الكبيرة حيث وجود history clean بدون تفرعات وبالتالي اسهل للقراءة

3/ git help

يستخدم فى حالة لو عايز اعرف تفاصيل اكثر عن command معين
مثال: `git merge – help // git help merge`
بيفتح documentation فيها شرح merge command

4/ git clean

بيمسح ال untracked files

5/ git shortlog

يبعرض الcommits حسب الauthor وده مفيد في حالة التيم ، لو عايز اعرف كل شخص عمل كام commits وايه هي الcommits دي

6/ git cherry-pick

**** معناها اني باخد التغييرات الموجود في commit معين وبضيفها في الbranch بتاعى**
**** السيناريو هنا : لو شغال على feature معين على branch منفصل , ومحتاج تغييرات خاصة**
بحل bugs من commit موجود على branch تاني مختلف ، من غير ما اضطر اعمل merge
**** هنا هعمل apply or copy لل commit ده داخل الbranch بتاعى**
**** مش بنقل الcommit ، فقط بعمل apply ل content داخل commit جديدة واضيفها على ال branch الخاص بيا**

****How to use this command :**

1/ Move to the branch where you want to apply the copied changes :

git switch my-feature

2/ Apply the commit to your current branch using the copied hash

git cherry-pick a1b2c3d

7/ git bisect

تستخدم في حالة لو ظهر bugs في البروجكت وعايز اعرف انهى commit كانت السبب في وجوده ، والجيت بيستخدم الجوريزم الbinary search علشان اوصل ل commit اللى سبب المشكلة

**** نستخدم الامر كالاتى :**

1/ git bisect start

2/ git bisect bad >> النسخة الحالية فيها مشكلة

3/git bisect good A >> متأكدة انها سليمة علشان يبدأ سيرش من عندها commit بحدد

4/ git bisect reset

8/ git prune

هى عبارة عن an internal housekeeping tool تستخدم لحذف الملفات او ال old commit او ال trees او branches الغير مستخدمة (unreachable objects) من الداتا بيز

**** سيناريو :** فى حالة استخدام rebase command

$A \rightarrow B \rightarrow C \rightarrow D' \rightarrow E'$

هنا مافيش حاجة بتشاوّر عليهم وبالتالي من ضمن D,E ال $\downarrow D \rightarrow E \Rightarrow$

unreachable data

فى الحالة دى ممكن نمسحهم باستخدام git prune

**** الداتا اللى اتمسحت cannot be recovered**

9/ git worktree

الفكرة هنا انى انتقل بين برانشز مختلفة فى نفس ال repo فى نفس الوقت بدون استخدام stash

****** عن طريق عمل working directly منفصل لكل برانش داخل البروجكت

****** السيناريو هنا مثلاً لو شغال على pay feature وحصل bug فى ال main بدل ما اعمل

stash للشغل الموجود فى برانش payment بستخدم الامر :

`git worktree add ../payment/pay-feature`

10/ git verify-commit

التأكد من ال signature الموجودة على ال commit مفيدة فى حالة المشاريع الكبيرة لو طبقنا

فكرة signed commit لكل يوزر فى التيم بحيث اقدر اتأكد من صحة commit معينة قبل

استخدامها خلال البروجكت

11/ git filter-repo

تستخدم لإعادة كتابة ال Git repository history وده مهم احياناً فى حالة لو عملت commit

لداتا Sensitive مسح الملف نفسه مش كفاية لان ممكن حد يرجعه من commit قديمة

فى الحالة دى ضرورى امسحه من ال history

****** مسح ال history بيغير ال IDs الخاصة ب commits وبالتالي ده ممكن يآثر لو الشغل

موجود remotly

12/ git grep

تستخدم في البحث عن كلمة او نص معين داخل البروجكت ، مثلا عايز اعرف استخدمت jwt فين ؟
(git grep jwt)

13/ git blame

بتعرف من خلالها مين كتب السطر ده في ملف ما
(git blame [author.js](#))
بترجع كل سطر في الكود مين اللي كتبه وايه ال hash commit الخاصة بيه وتاريخ التعديل

.....

Leetcode links:

My account:

<https://leetcode.com/u/ayaatea/>

Submission link:

<https://leetcode.com/problems/two-sum/submissions/2117897356>

.....

Linkedin posts:

Squash post :

https://www.linkedin.com/posts/aya-atea-6766a831a_aepaepaevaetabrgita_braedaev-git-github-activity-7497636333100748800-M1ky?utm_source=social_share_send&utm_medium=android_app&rcm=ACoAAFDni7oBivacE8AEf2OpmTXXS_P3KhxDwMI&utm_campaign=copy_link

Merge vs rebase post :

https://www.linkedin.com/posts/aya-atea-6766a831a_aepaepaevaetabrgita_braedaev-git-github-activity-7497625008496689152-Xnxr?utm_source=sha

[re&utm_medium=member_android&rcm=ACoAAFDni7oBivacE8AEf2OpmTXXS_P3KhxDwMI](#)

Cherry pick post :

[https://www.linkedin.com/posts/aya-atea-6766a831a_aepaepaevaetabrgita
braedaev-git-github-activity-7497643254100738048-2KP3?utm_source=sh
are&utm_medium=member_android&rcm=ACoAAFDni7oBivacE8AEf2Op
mTXXS_P3KhxDwMI](#)